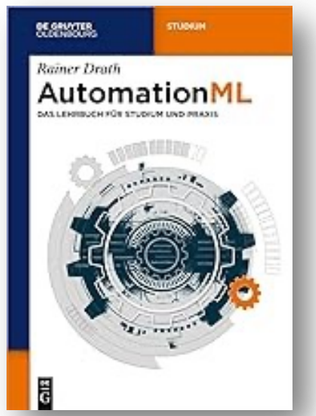
FHV
Vorarlberg University
of Applied Sciences

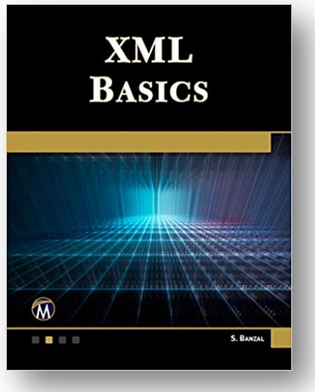
Technical Information Systems

7 Object-relational Impedance Mismatch, XML – Extended Markup Language

Literature



- [Drat22]
  Drath, Rainer: AutomationML: das Lehrbuch für Studium und Praxis. Berlin: De Gruyter Oldenbourg, 2022 — ISBN 978-3-11-078293-6 (https://go.exlibris.link/QFs5j0zV)



- [Banz20]
  Banzal, S.: *XML basics*. Duxbury: Mercury Learning and Information, 2020 — ISBN 978-1-68392-546-0 (https://go.exlibris.link/ZXS0yyMY)

- [Tyso22]
  Tyson, Matthew: What is JPA? Introduction to Java persistence. URL https://www.infoworld.com/article/3379043/what-is-jpa-introduction-to-the-java-persistence-api.html — InfoWorld

6. ORM, XML

1. Object-relational Impedance Mismatch

The problems between object orienation and relational systems

- Granularity Problem

  Granularity is the extent to which a system is composed of smaller components and the relative size of it's elements

  In an object model, classes can consist of other classes that again reference other classes – the granularity has no limitation

  Relational systems only have tables and columns – a deeper structure is not possible.

  Example:

```
Public class Address {
   private String street;
   private City city;
   private State state;
   private Integer zipcode;
   }
```

```
Public class Cinema {
   private string name;
   private Adress address;
}

Public class User{
   private string firstName;
   private string lastName;
   private Adress address;
}
```

[Shas21]

The problems between object orienation and relational systems

- Inheritance Problem

  The object oriented paradigm supports type inheritance.
  E.g. a User can be an abstract class and could have different child classes such as Customer and Administrator, both defining a user of the Ticket booking system but with different roles and privileges.
  This also adds polymorphic behaviour for the object instances.

```
Public abstract class User {
   private String username;
   private String password;
   private Set<Role> roles;
   }
```

```
Public class Customer extends User {
   private Set<Booking> bookings;
}

Public class Admin extends User {
   private String designation;
}
```

[Shas21]

The problems between object orienation and relational systems

- Identity Problem

  Object orientation has no equivalent to the primary key – particularly surrogate key. Object identification uses other means.
- Association Problem

  Association refers to the concept of entity(table) relationships in relational systems. In object orientation association is the relationship between objects using object references. There is no equivalent to the integrity check with foreign keys in obj. orient.
- Data Navigation Problem

  Object orientation allows simple navigation through referenced objects with the reference point (e.g. cinema.getAddress().getCity()). There is no corresponding concept in relational db.

[Shas21]

Why do we need the connection? Persistence and data access

- In almost all cases, applications must handle data. Data that should not vanish when the application is stopped, but can be restored when re-establishing the programme.
- Object persistence means individual objects can outlive the application process; they can be saved to a data store and be re-created at a later point in time.
- When we talk about persistence in Java, C# or similar, we're normally talking about mapping and storing object instances in a database using SQL.

- Furthermore, we often need access to data that was not created in the app at hand

[BKGB16:4]

Strategies to overcome the difference between object orientation and relational systems

1. Using an object-oriented database management system (OODMBS)
2. Serialising the data. Converting the data structure into a string (e.g. XML or JSON) or a binary format
3. Build an object-relational database system, which adds object-oriented features to a relational database system.
4. Object-relational mapping: Using libraries for the programming system to access data in a rather object oriented way.

[SiKS20:377] [Kofl22:81-82]

Strategies to overcome the difference between object orientation and relational systems

1. Using an OODMBS
   Unfortunately, with the exception of a few niche applications, all attempts proved to be non-marketable.

2. Serialising the data. Converting the data structure into a string (e.g. XML or JSON) or a binary format
   This approach degrades the DBMS to a file system without any significant search capabilities. There are only rare use cases where this approach is reasonable.

[Kofl22:81, 99-103]

Strategies to overcome the difference between object orientation and relational systems

3. Build an object-relational database system, which adds object-oriented features to a relational database system.

   Since 1999, the SQL-Standard includes some features to serve OO:

   - Definition of new data types as counterpart to classes
   - Definition of simple functions as counterpart to methods
   - Inheritance of types and tables
   - References to other objects
   - Processing of enumerations: Arrays, lists, sets

   Unfortunately, these attempts fall short to the demands of programming and were already obsolete because programmers felt comfortable with the ORM-solution described on the next pages

[SiKS20:377-381] [Kofl22:101] https://learn.microsoft.com/en-us/sql/t-sql/statements/create-type-transact-sql?view=sql-server-ver15

Strategies to overcome the difference between object orientation and relational systems

4. Object-relational mapping

Object-relational mapping (ORM) systems allow a programmer to define a mapping between tuples in database relations and objects in the programming language.

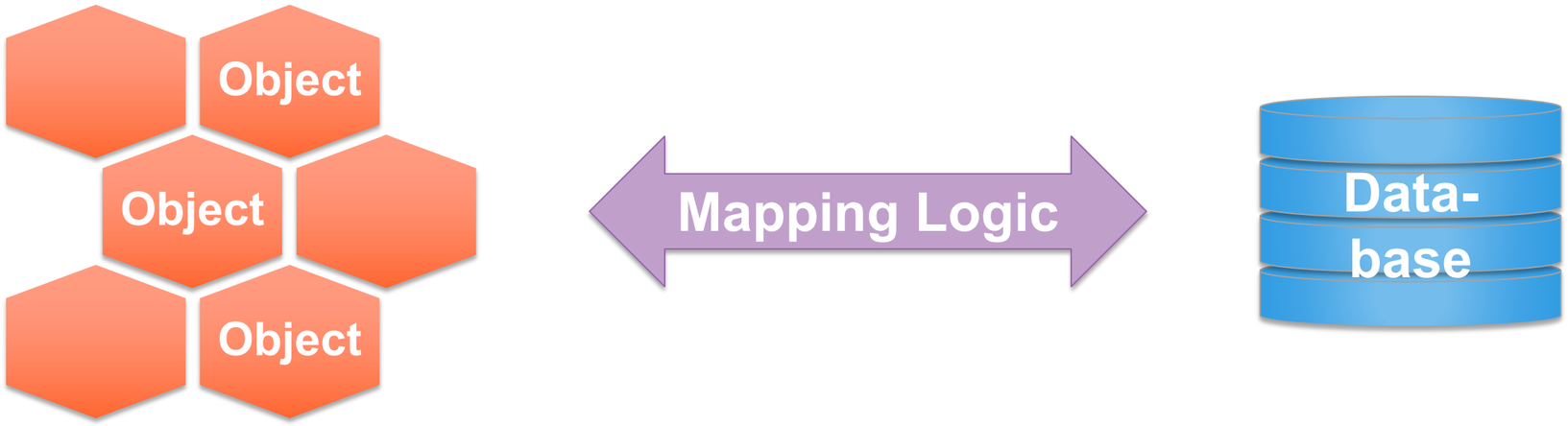
Objects are retrieved from the database on selection conditions on attributes. The selection result is converted into objects to be used in the OO code.

The program can update retrieved objects, create new objects, or specify that an object is to be deleted, and then issue a save command; the mapping from objects to relations is then used to correspondingly update, insert, or delete tuples in the database.

[SiKS20:381]

Object-relational mapping (ORM)

- Modern programming is typically object-oriented
- Modern databases are relational
- ORM converts relational data structures into object-oriented information



- Often used ORM-Frameworks are Core Data (Swift), SQLAlchemy (Python), Hibernate (Java), Lavarel (PHP) and LINQ to SQL (C#).

Object-relational mapping (ORM)

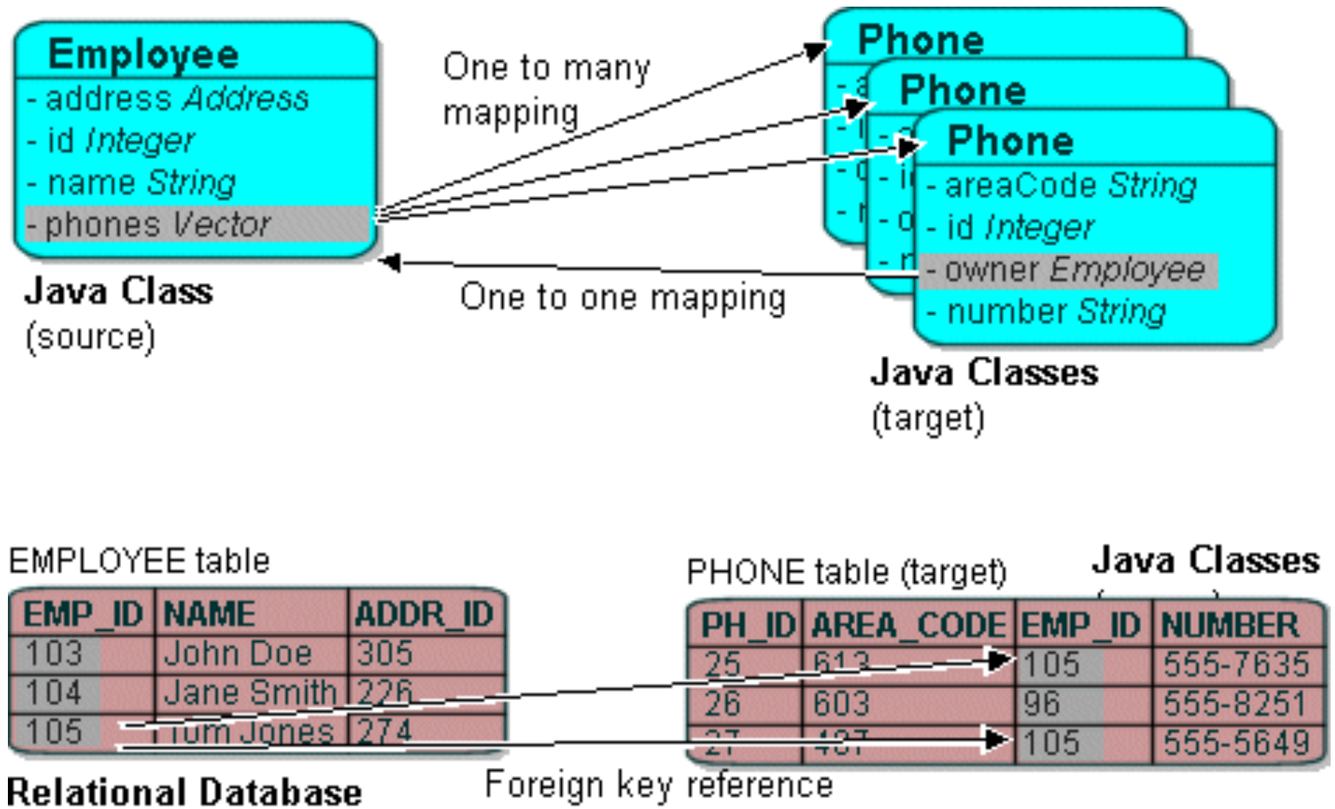
- Goals of ORM
  - object-oriented access to persistent data
  - transparent loading and saving of persistent data
  - simple exchange of the subjacent data management
  - increase of the performance through buffering of data of the data base in the main memory objects

- Performance is one of the main issues of ORM and needs to be treated carefully. Loading objects one by one leads to many data base accesses. Loading everything leads often to too much data. The programmer needs to decide what to do while implementing ORM in his/her application.

[Kudr15]:177-188]

Object-relational mapping (ORM)

- The easiest form of mapping creates one class for each table with the fields of the table corresponding to attributes (and verse vicar)



- Unfortunately, this solution is not always feasible.

image: https://wiki.eclipse.org/EclipseLink/UserGuide/JPA/Basic_JPA_Development/Mapping/Relationship_Mappings/Collection_Mappings/OneToMany

Java Persistence API (JPA)

- The Java Persistence API (JPA) is a Java specification for accessing, persisting, and managing data between Java objects / classes and a relational database.
- The mapping needs to be declared first
- snippet:

```
@Entity
@Table (name="Article")
public class Article implements Serializable {
    private String anr;
    private String description;
    public Article()
…
```

- Example for access to the data

```
Article a=em.find(Article.class, anr);
a.setDescription(a.getDescription() + "_OLD";
em.flush()
```

KUDRAS, THOMAS: *Taschenbuch Datenbanken*. 2. München: Carl Hanser Verlag GmbH & Co. KG, 2015 — ISBN 978-3-446-43508-7 P 177-188

Java Persistence API (JPA) - example

Example: Function to get a list of authors filtered by last name

```
import javax.persistence.EntityManager;
import javax.persistence.TypedQuery;
...
public List<Author> getAuthorsByLastName(String lastName) {
    String queryString = "SELECT * FROM Author a " + "WHERE LOWER(a.lastName)
                         = LOWER(:lastName)";
    TypedQuery<Author> query = getEntityManager().createQuery(queryString,
                                Author.class);
    query.setParameter("lastName", lastName);
    return query.getResultList();
}
```

To use Microsoft SQL in Java use Hibernate: https://hibernate.org/orm/

6. ORM, XML

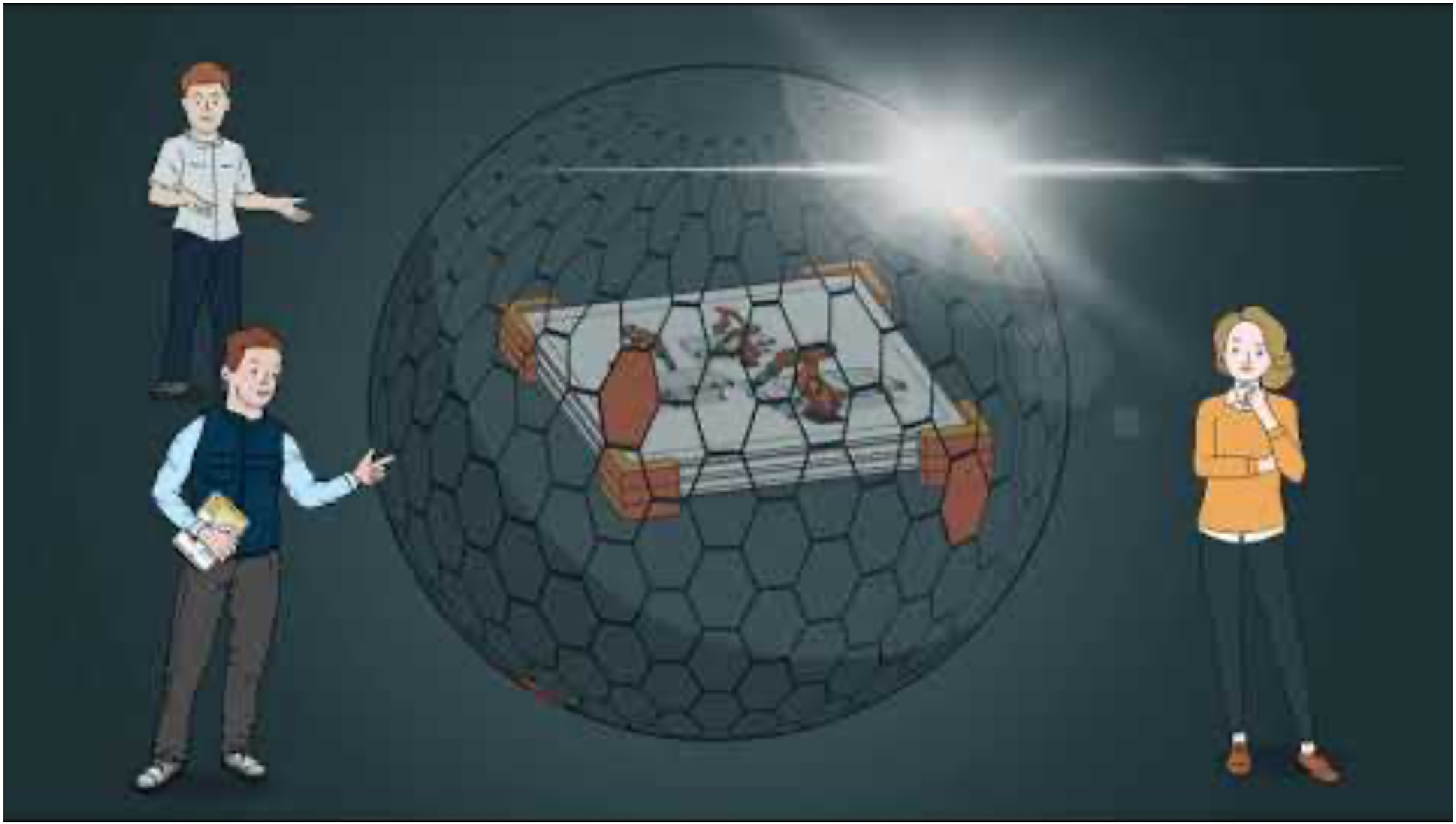
1. ORM – Object Relational Mapping
2. XML – Extended Markup Language

WHY XML? – An example

- ETIM (*ElektroTechnisches InformationsModell*) is an open standard for the unambiguous grouping and specification of products in the technical sector (namely electrotechnical sector) through a uniform classification system.
- Goal is the electronic exchange of product data for technical products, to enable the electronic trading of these products
- ETIM – like many other initiatives – uses the XML format BMEcat as data model.
- It is a living, active standard: since December 2022 Version 9.0 is released
- More information to ETIM International https://www.etim-international.com/

Video: https://www.youtube.com/watch?v=zudr-A6jSJo https://en.wikipedia.org/wiki/ETIM_(standard)

Why XML – another example – Automation ML



Video: https://youtu.be/I1Pk4gj5Fsg

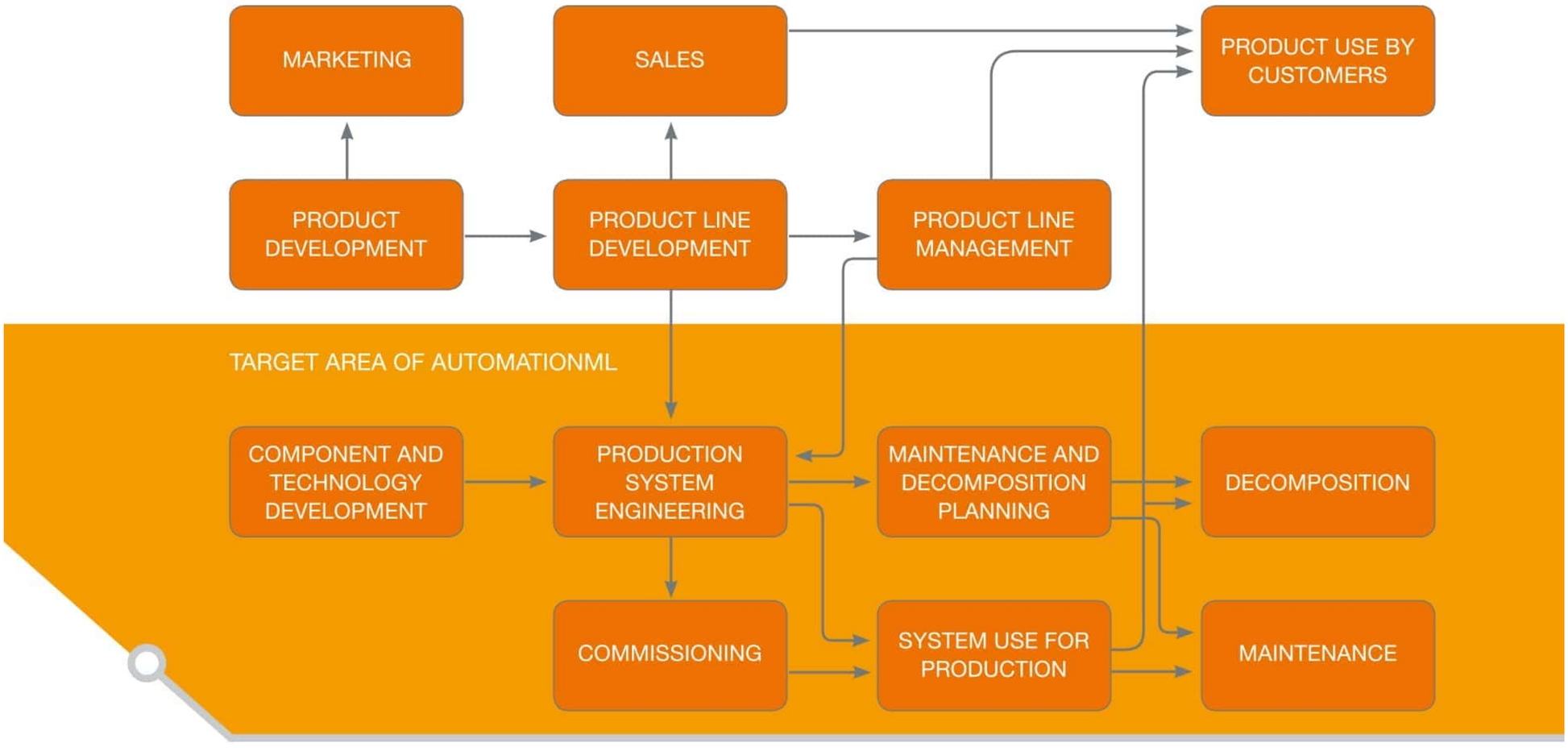
Why XML – another example – Automation ML

- AutomationML is a comprehensive XML based object-oriented data modeling language.
- It allows the modelling, storage and exchange of engineering models covering a multitude of relevant aspects of engineering.
- AutomationML has a lean and distributed file architecture. It does not define any new file format but combines existing established XML data formats:
  - object topologies including hierarchies, properties and relations of objects: CAEX according to IEC 62424
  - geometries and kinematics of objects: COLLADA 1.4.1 and 1.5.0 (ISO/PAS 17506:2012)
  - discrete behavior of objects: PLCopen XML 2.0 and 2.0.1; in addition, IEC62714-4 will allow the usage of IEC61131-10

https://www.automationml.org/about-automationml/automationml/

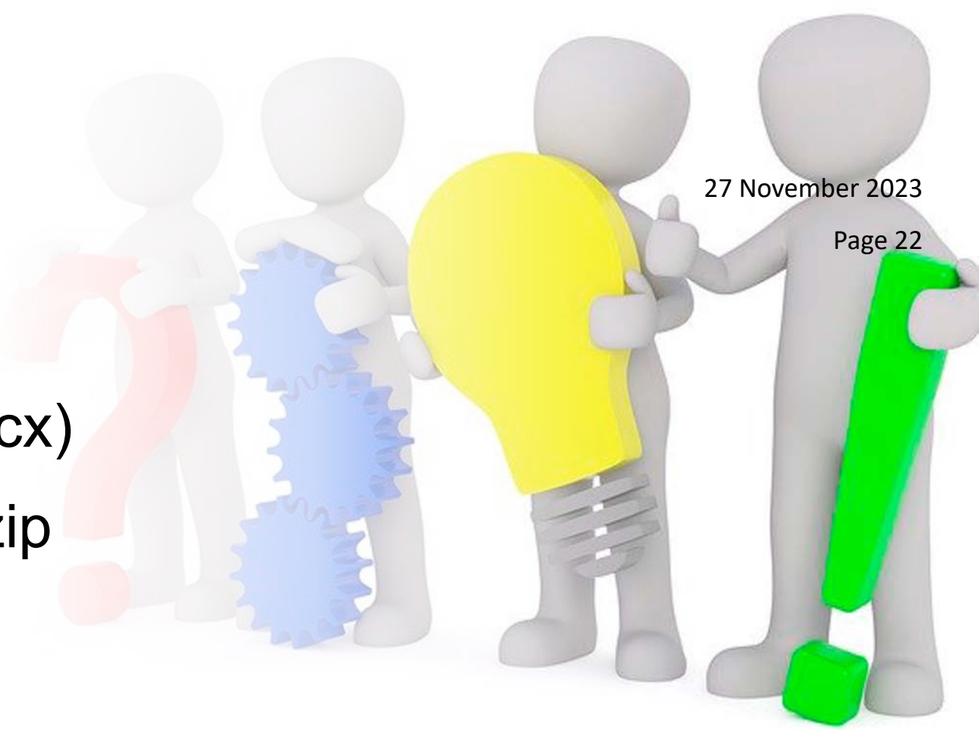
Why XML – another example – Automation ML

- Target area of Automation ML



https://www.automationml.org/about-automationml/automationml/

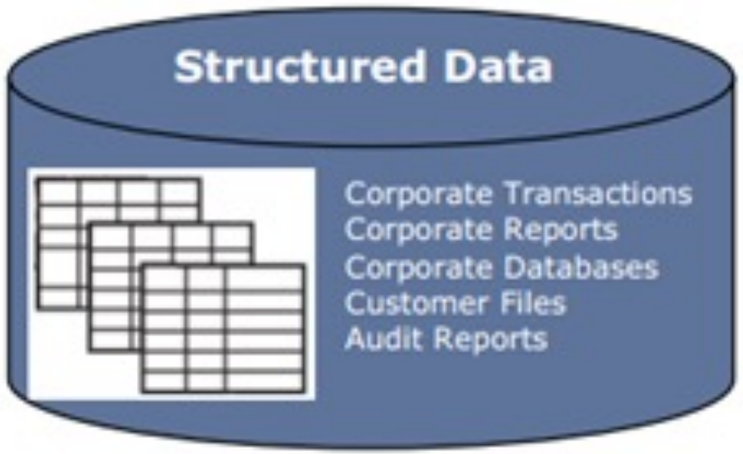
Have some fun with a word file



- Take a SHORT Word document (e.g. xml_demo.docx)
- Rename the document in your file manager - add .zip (e.g. xml_demo.docx.zip)
- Expand the zip-file
- Explore the result

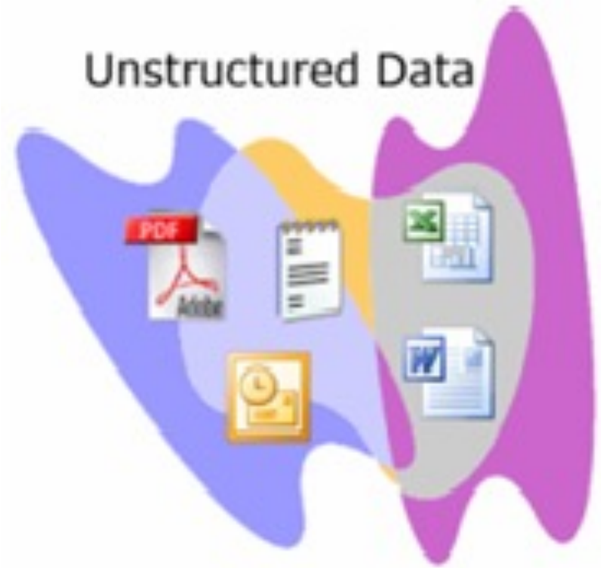
- What do you see?

Structured, semistructured and unstructured Data



Semi- structured data

```
<Sensor>
  <name>Sensor 193</name>
  <attributes>
    <Attribute>
      <name>Alpha</name>
      <x>101</x>
      <y>20031</y>
    </Attribute>
    <Attribute>
      <name>Beta</name>
      <x>243</x>
      <y>3037</y>
    </Attribute>
  </attributes>
  <scale>1</scale>
</Sensor>
```

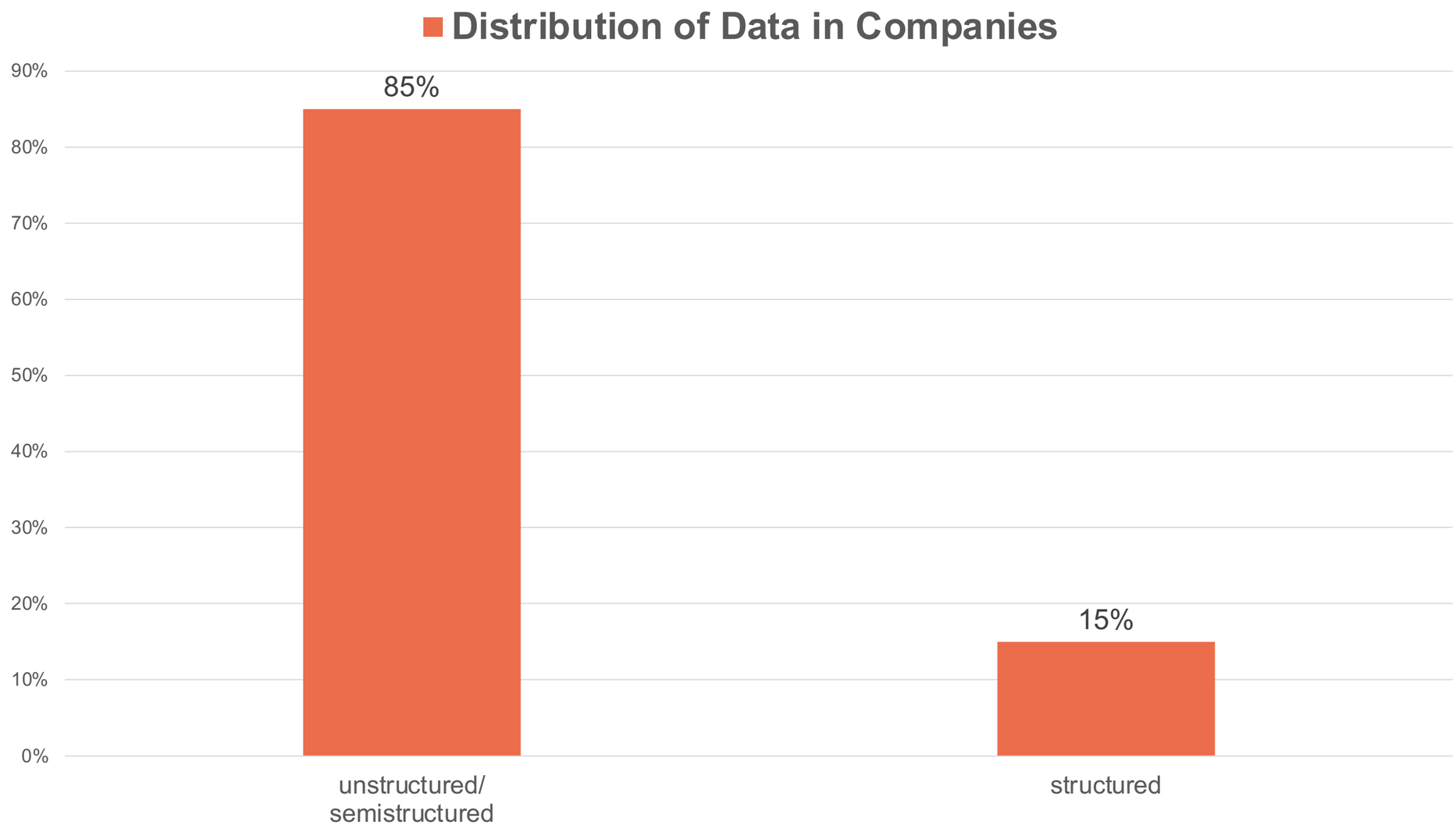


Source: http://bhavnaober.blogspot.de/2015/02/data-warehouse-current-scenario.html

Structured, semistructured and unstructured Data

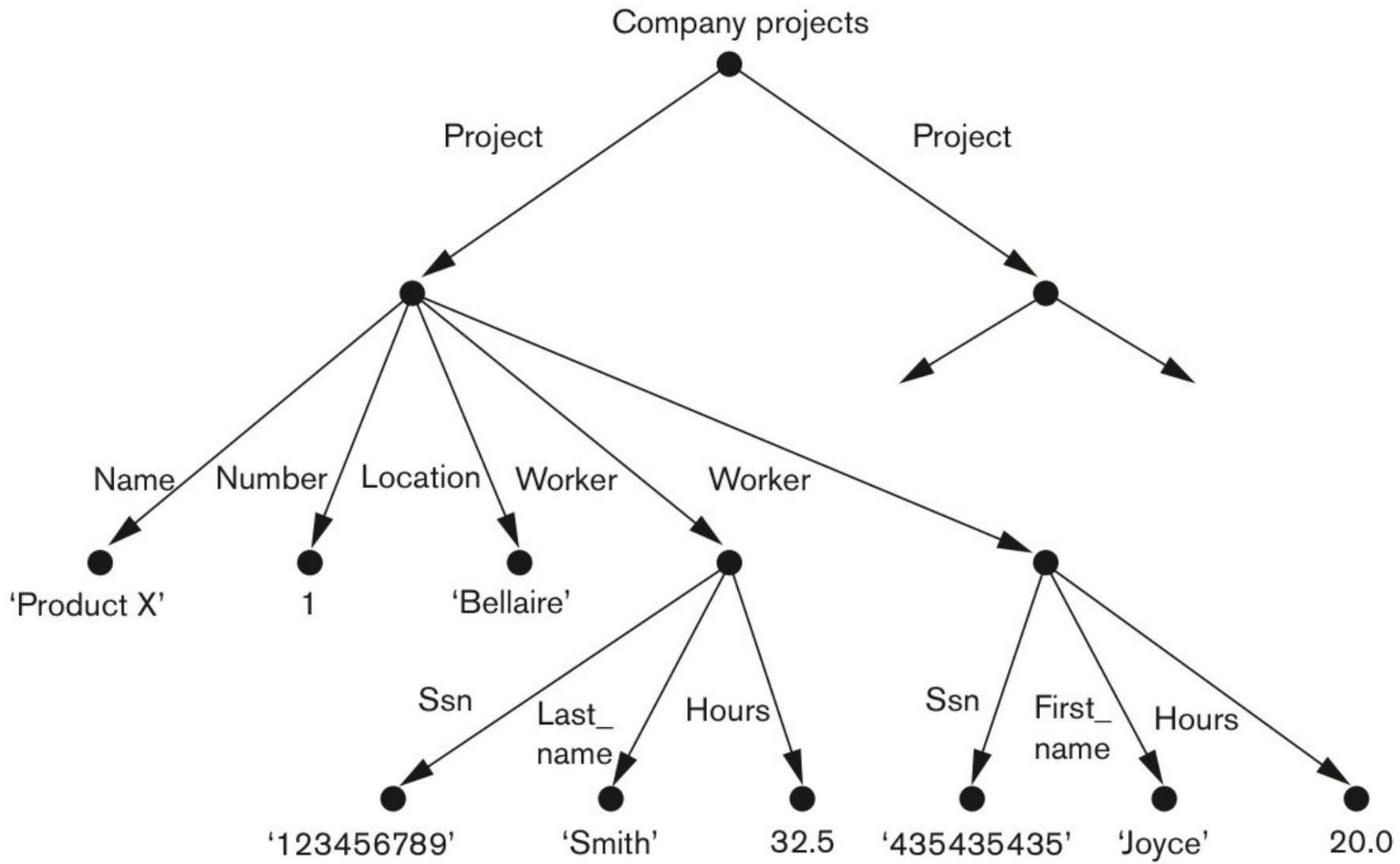
- **Structured Data**
  Data represented in a strict format
  e.g. simple content of well designed relational Databases structured in tables
- **Semistructured Data**
  Semistructured Data is stored in an unstructured data source but has an inherent structure. There is a certain structure but not all data follows the same rules
- **Unstructured Data**
  There is no or no consistent structure in the data. E.g. a text file with its content or a web page with unstructured HTML (consisting of content and mixed tags for structure and format)
  → The data it not readable (interpretable) by machines

Structured, semistructured and unstructured Data



[Geis11] P496

Semistructured Data represented as a graph



[ElNa16] P457

HTML as Example for Unstructured or Structured Data

Examples of logical tags

- <cite> Defines citation. It displays the text in italic format
- <code> Defines the piece of computer code. It displays the text in Courier
- Defines strong text i.e. show the importance of the text (shown bold)
- <strike> It is an editing markup that tells the reader to ignore the text passage.

Examples of physical tags

- <i> An Italic tag is used to define a text with a special meaning.
- <b> Shows the text with a bold font-style
- <p style="font-family:'Courier New'"> Defines the font shown in the praragraph
- <h3 text-decoration: line-through;> Shows the following heading striked out

https://www.geeksforgeeks.org/difference-between-physical-and-logical-tags/

Why should we have XML?

- Possible competitors:
  - HTML
  - Data bases
  - Document formats like *.doc or *.rtf

- Problem of HTML:
  HTML is unstructured and inflexible

- Problem of classic data bases:
  Data bases can only describe structured data

- Problem of RTF and Word-DOC (Word 97 or older):
  Both are no standards and can't describe structure

Documents

Documents have

- Content ← Their payload – the information within
- Structure ← How they are composed
- Format ← What defines the visual result
- Metadata ← What we know additionally

Why is it so troublesome to store conventional documents in traditional databases?

Reasons:

- Inner structure
- Hierarchical dependencies
- Formatting
- Arrangement
- Extremely alternating length
- Extremely alternating composition

XML

- XML is a solution for these problems.
  It is a generally approved description standard, that is capable to describe "tree like" objects

- It has similarities with HTML, but is much more flexible

- The standard is active since 1998 and is very stable.
  The current version is 1.1. but 1.0. is still widely used

- The standard is supervised by the World Wide Web Committee

- XML's ancestor is SGML – the Standard Generalized Markup Language (HTML is based on SGML)

XML - advantages

- XML can perform presentation, communication, and storage of data.
- Ease of data exchange
  - The standard is firm but simple, storage is efficient
  - The data is not stored in a proprietary format and can even be edited with plain text editors
- Self-explanatory data
  - The data can be read by humans without additional tools
- Structured and integrated data
  - Not only data but also its structure can be defined in XML
  - Semantic rules demand a clear structure
- Customization of markup languages
  - It is technologically simple to create a standardized markup language on the basis of XML – like ETIM
- It is the foundation technology for web services

[BaLn20], [LaLa20:224]

Properties of Documents

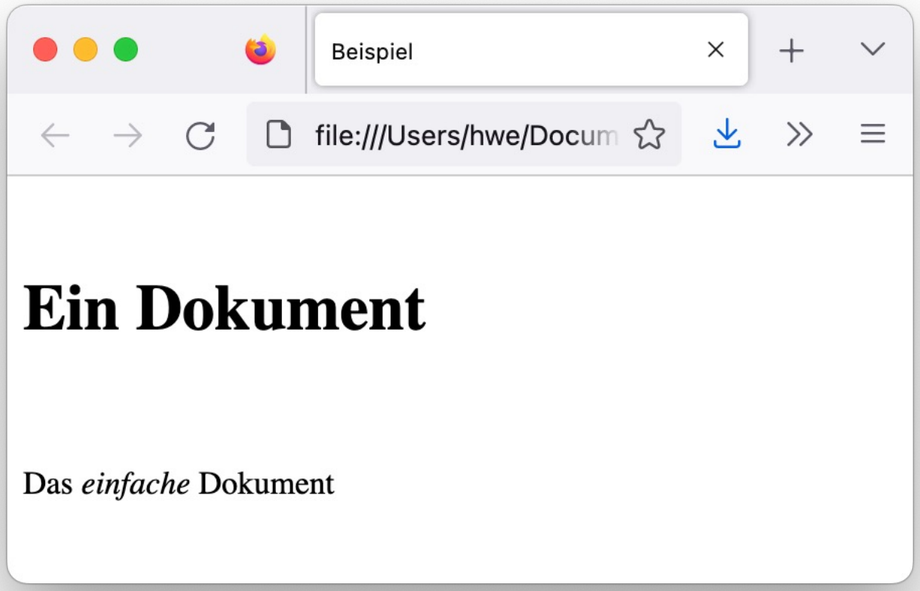
- **Content → what they contain below the line**
  - Different content building blocks e.g. text, image, tables, lists...
  - Building blocks are interrelated
  - Arrangement is relevant
  - Highly alternating lengths are possible
  - Highly alternating compositions are possible

- **Structure → what represents their composition**
  - Documents are hierarchical → tree structure
  - Documents can be arbitrarily deep structured
  - The structure may be highly irregular

- **Format → what describes their look**

- **(Meta data) → what is known additionally**

(X-)HTML as an example of XML

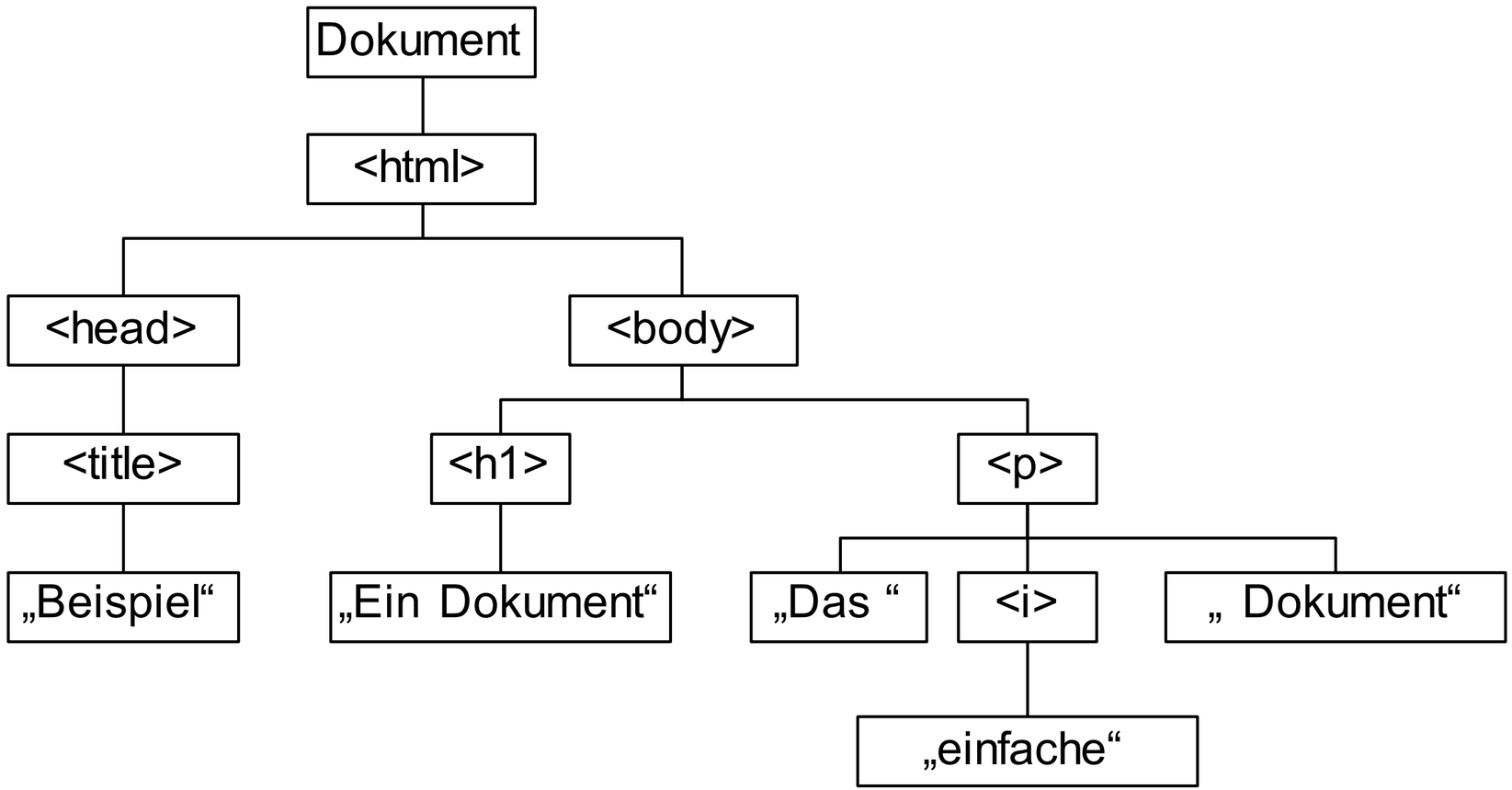



```
<html>
  <head>
    <title>Beispiel</title>
  </head>
  <body>
    <h1>Ein Dokument</h1>
    <p>Das <i>einfache</i>
  Dokument</p>
  </body>
</html>
```

(X-)HTML as an example of XML


```
<html>
  <head>

     <title>Beispiel</title
  >
  </head>
  <body>
     <h1>Ein Dokument</h1>
     <p>Das <i>einfache</i>
  Dokument</p>
  </body>
</html>
```



Structure of XML

XML is a markup language.

Content, structure, format and meta data are described as text and can be in one document

XML is case sensitive. Attribute values must be quoted

<elementname> … **</elementname>**

Tag

- <element **attribute="information"**> …

Attributes in XML provide additional information to elements

XML – Example code


```
<?xml version= "1.0"
  standalone="yes"?>
<Projects>
  <Project>
    <Name>ProductX</Name>
    <Number>1</Number>
    <Location>Bellaire</Location>
    <Dept_no>5</Dept_no>
    <Worker>
      <Ssn>123456789</Ssn>
      <Last_name>Smith</Last_name>
      <Hours>32.5</Hours>
    </Worker>
    <Worker>
      <Ssn>453453453</Ssn>
      <First_name>Joyce</First_name>
      <Hours>20.0</Hours>
    </Worker>
  </Project>
  <Project>
    <Name>ProductY</Name>
    <Number>2</Number>
    <Location>Sugarland</Location>
    <Dept_no>5</Dept_no>
    <Worker>
      <Ssn>123456789</Ssn>
      <Hours>7.5</Hours>
    </Worker>
    <Worker>
      <Ssn>453453453</Ssn>
      <Hours>20.0</Hours>
    </Worker>
    <Worker>
      <Ssn>333445555</Ssn>
      <Hours>10.0</Hours>
    </Worker>
  </Project>
...
</Projects>
```

Structure of XML

Elements

<elementname> … **</elementname>**

- Elements are defined to describe the **meaning** of the containing data. → The data can be read by machines
- **Schemas** are documents that define the elements, give them a **semantic meaning**
- Schema based XML-files can be variously exchanged

- **Simple elements** contain data values
- **Complex elements** are constructs of other hierarchical structured elements

Structure of XML

Attributes

<element **attribute="information"**> …

- Attributes describe properties and characteristics of their corresponding elements (tags)
- Sometimes attributes contain values of simple data elements, but this is typically deprecated
- The only exception to that rule are cross-references to other elements alike foreign keys. These methods are the only possibility to bypass the strict hierarchical structure of XML

Usage of Cross-References in XML


```
<class id="ENGL6004">
  <title>From Here to Eternity</title>
  <teacher id="T31330">
    <name>Margaret Doornan</name>
    <position>Associate Professor</position>
  </teacher>
  <students>
    <student id="S50245">
      <name>Maggie Trudeau</name>
      <year>Junior</year>
      <status>part-time</status>
    </student>
    <student id="S87901">
      <name>John Atterly</name>
      <year>Senior</year>
      <status>full-time</status>
    </student>
    ...
</students>
</class>
```

```
<classes>
    <data:class id="ENGL6004">
      <title>From Here to Eternity</title>
      <ref:teacherRef ref="T31330"/>
      <students>
        <ref:student ref="S50245"/>
        <ref:student ref="S87901"/>
        <ref:student ref="S19272"/>
        <ref:student ref="S48984"/>
      </students>
    </data:class>
    <data:class id="HIST6010">
      <title>The You Decade</title>
      <ref:teacher ref="T72100"/>
      <students>
        <ref:student ref="S60912"/>
        <ref:student ref="S87901"/>
        <ref:student ref="S84281"/>
        <ref:student ref="S44098"/>
      </students>
    </data:class>
    …
```

Ref: https://msdn.microsoft.com/en-us/library/ms950811.aspx

Well Formed

An XML-Document is well formed, when it follows the syntax of the XML 1.0 Specification. Notably:

- It starts with the XML-Declaration
- It is strictly follows the tree data model
- It consists only of Unicode characters
- It is syntactically correct

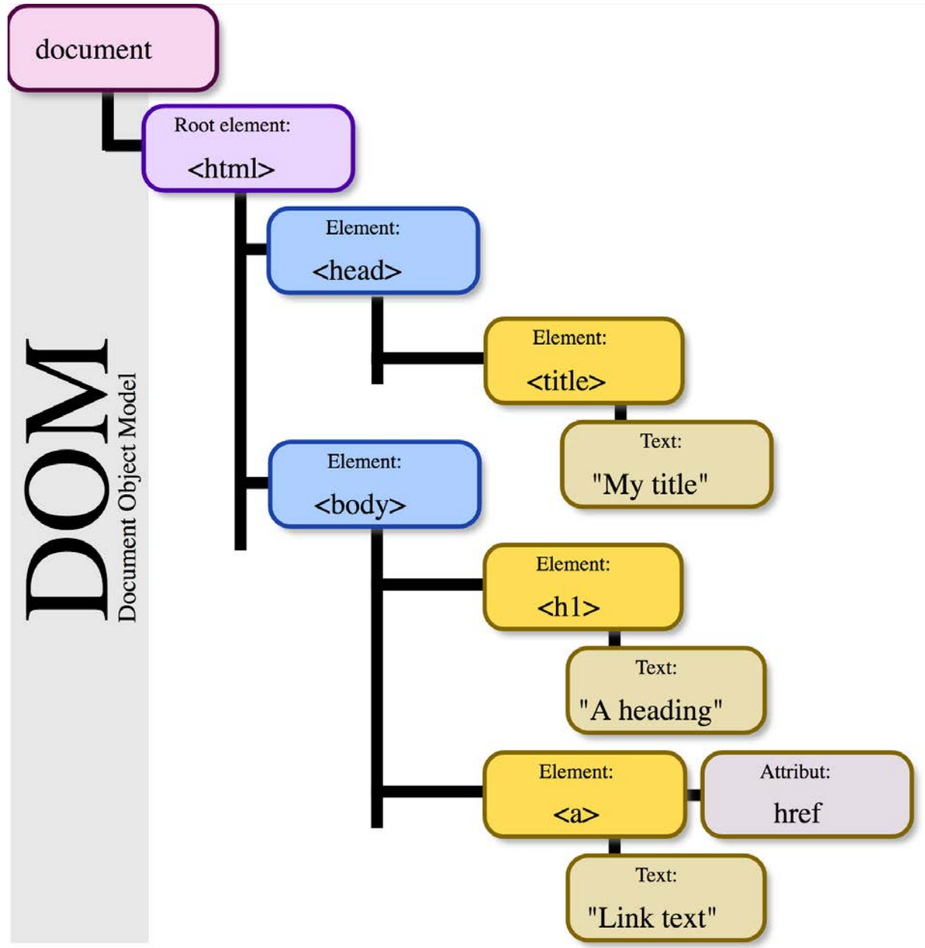
➢ Only well formed files are XML files

➢ Only those are permitted for further processing, others must be refused

```
<?xml version= “1.0” standalone=“yes”?>
<Projects>
  <Project>
    <Name>ProductX</Name>
    <Number>1</Number>
    <Location>Bellaire</Location>
    <Dept_no>5</Dept_no>
    <Worker>
      <Ssn>123456789</Ssn>
      <Last_name>Smith</Last_name>
      <Hours>32.5</Hours>
    </Worker>
    <Worker>
      <Ssn>453453453</Ssn>
      <First_name>Joyce</First_name>
      <Hours>20.0</Hours>
    </Worker>
  </Project>
</Projects>
```

XML Processors

- An XML document is read by a processor or parser. Via the corresponding API the data can be imported into an application for further use
- The **Document Object Model (DOM)** and its appendant API allows manipulating the XML tree model. E.g. It is used from programming languages for web pages and allows changing the appearance of every object of the side.
- **SAX (Simple API for XML)** allows working on very large files, because it is able to parse streaming data and to work on tree segments.
- **Pull Parser** like **StAX** read the XML data iteratively and can navigate through a complex, streamed files



https://en.wikipedia.org/wiki/XML#Programming_interfaces Imagesource: https://upload.wikimedia.org/wikipedia/commons/5/5a/DOM-model.svg

The "career" of XML

- XML was created and planed for "tree like" structures but it developed quickly to a widely used standard for:
  - Cross-platform information description
  - Exchange of information between applications

- Reasons
  - Technical: The tree-structure is very flexible. Notably tables can easily be represented in the syntax
  - Practical: There is a huge demand for information exchange

Types of XML Documents

- **Data-centric XML documents.**
  Tend to have small data items. It is possible to export them from a relational database. They are used as exchange or communication format. They are often based on a standardized schema
- **Document-centric XML documents.**
  Comprise often of large amounts of text. There are few or no structured data elements in these documents.
- **Hybrid XML documents.**
  These documents may have parts that contain structured data and other parts that are predominantly textual or unstructured. They may or may not have a predefined schema.

[ElNa16] 461

Types of XML Documents

- **Examples**

Data oriented	Document oriented

<invoice>
  <orderDate>1999-01-21</orderDate>
  <shipDate>1999-01-25</shipDate>
  <billingAddress>
  <name>Ashok Malhotra</name>
  <street>123 Microsoft Ave.
  </street>
  <city>Hawthorne</city>
  <state>NY</state>
  <zip>10532-0000</zip>
  </billingAddress>
  <voice>555-1234</voice>
  <fax>555-4321</fax>
</invoice>
	<memo importance='high'
	  date='1999-03-23'>
	  <from>Paul V. Biron</from>
	  <to>Ashok Malhotra</to>
	  <subject>Latest draft</subject>
	  <body>
	  We need to discuss the latest
	  draft <emph>immediately</emph>.
	  Either email me at <email>
	
	mailto:paul.v.biron@kp.org</email>
	  or call <phone>555-9876</phone>
	  </body>
	</memo>

Source: W3C

DTD and XML Schema

- To really make use of XML-data, the elements, attributes and their structure must be defined

- Information for different application areas need description tools that are tailor made to their tasks

- By now there are numerous descriptions for diverse purposes in XML

```
…
<math>
  <mo>&int;</mo>
    <mfenced separators=''>
      <mn>5</mn>
      <mi>x</mi>
      <mo>+</mo>
      <mn>2</mn>
      <mi>sin</mi>
      <mfenced separators=''>
        <mi>x</mi>
      </mfenced>
    </mfenced>
  <mi>dx</mi>
</math>
```

$$\int (5x + 2\sin(x))\,\mathrm{dx}$$

Details to MathML: https://www.w3.org/TR/REC-MathML/

DTD and XML Schema

- An XML schema classifies an XML document and delivers a syntactical description of its structure and content

- There are special languages to fulfil this task
  The two most important are
  - **Document Type Definition (DTD)**
    Derives from SGML and is part of the XML standard. Because it lacks flexibility it is mostly only used in the field of publishing
  - **XML Schema (XSD)**
    Today the most important description language. It uses the same syntax as XML documents

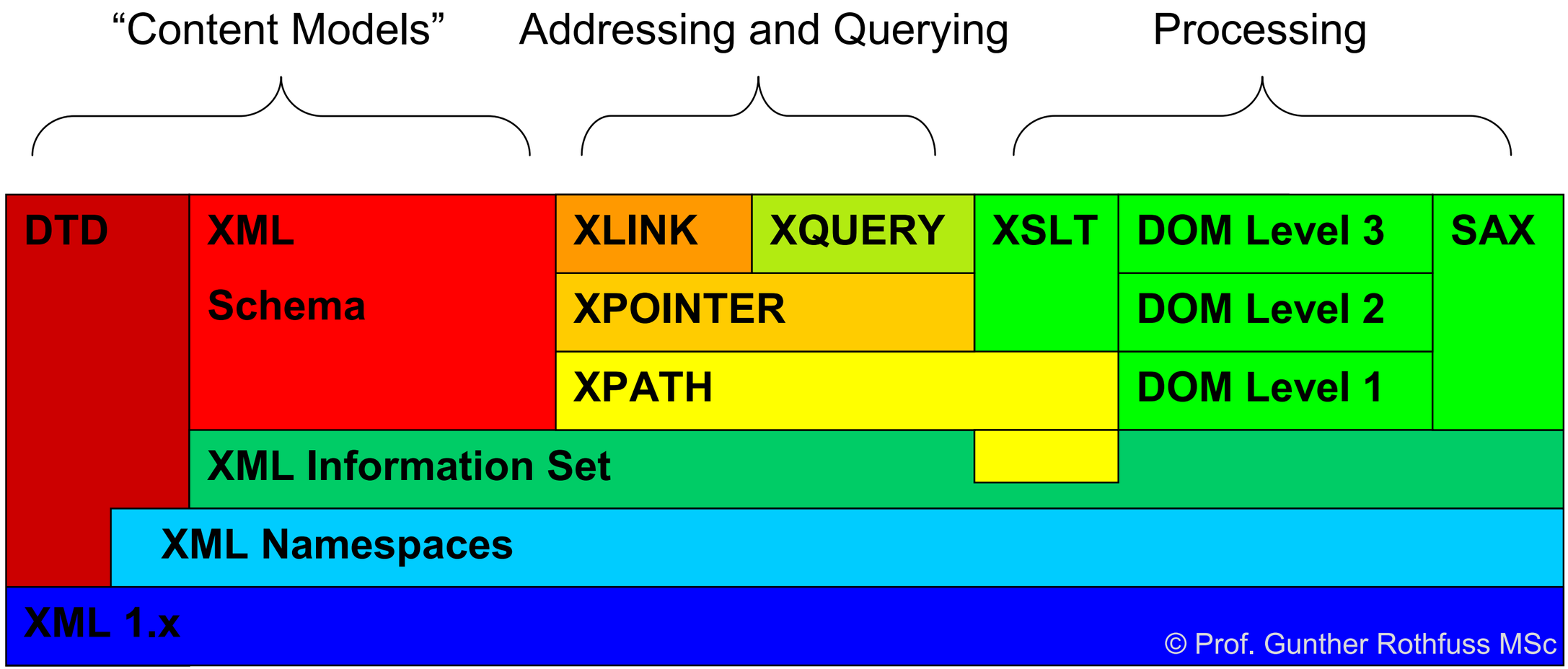
- An XML document that is well formed and that follows an XML schema is **valid**

XML Schema example

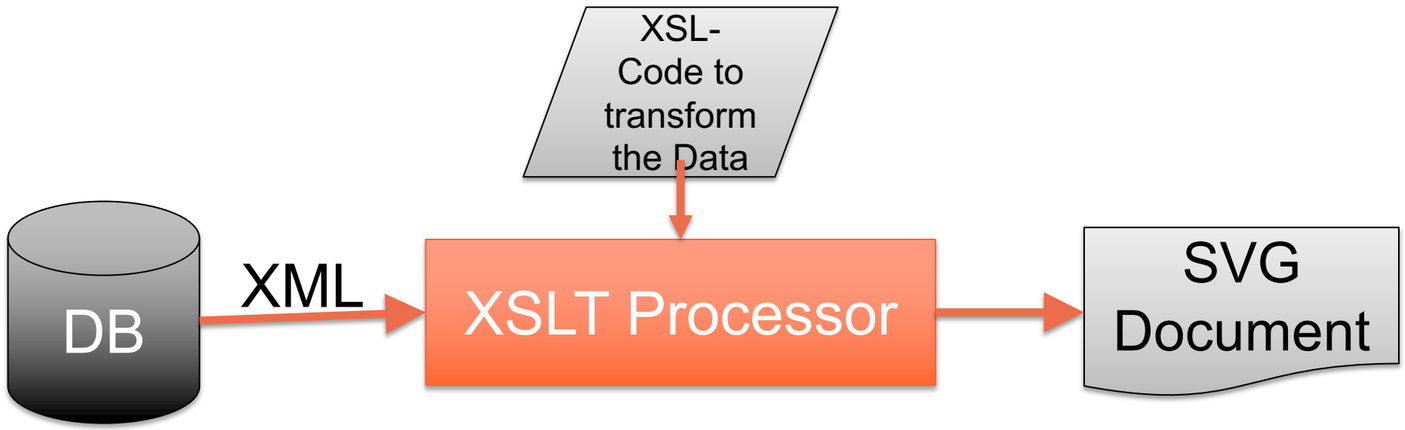

```
<?xml version="1.0" encoding="UTF-8" ?>
<xsd:scheman XMLns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:annotation>
    <xsd:documentation xml:lang="en">Company Schema (Element Approach) - Prepared by Babak
    Hojabri</xsd:documentation>
  </xsd:annotation>
<xsd:element name="company">
  <xsd:complexType>
  <xsd:sequence>
    <xsd:element name="department" type="Department" minOccurs="0" maxOccurs= "unbounded" />
    <xsd:element name="employee" type="Employee" minOccurs="0" maxOccurs= "unbounded">
                  <xsd:unique name="dependentNameUnique">
                       <xsd:selector xpath="employeeDependent" />
                       <xsd:field xpath="dependentName" />
                  </xsd:unique>
    </xsd:element>
    <xsd:element name="project" type="Project" minOccurs="0" maxOccurs="unbounded" />
  </xsd:sequence>
  </xsd:complexType>
  ...
```

The full example: [ElNa16] 467-469

The XML family



From Data to svg (scalable vector graphic)

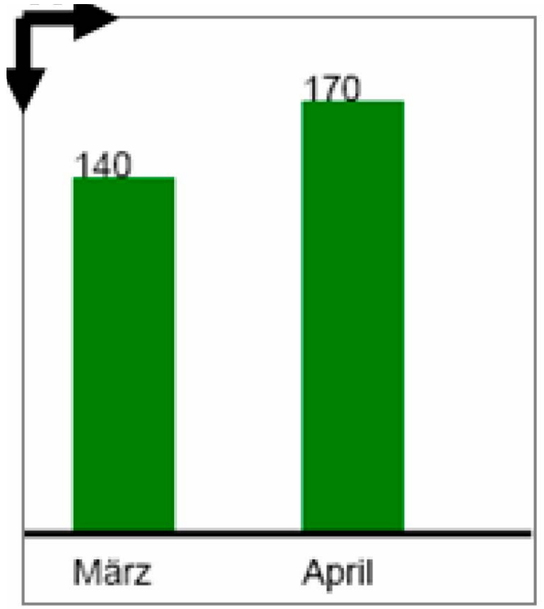


[ScEr07]

Bodeninformationssystem Bayern BIS-BY

Zeit	Wert
März	140
April	170

```
<?xml version="1.0„
        encoding="UTF-8"?>
<daten>
   <datensatz>
      <zeit>März</zeit>
      <wert>140</wert>
   </datensatz>
   <datensatz>
      <zeit>April</zeit>
      <wert>170</wert>
   </datensatz>
</daten>
```



Data

XML

SVG

[ScEr07]

Extensible Stylesheet Language (XSL) Transformation instruction

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<xsl:output encoding='UTF-8'/>
<xsl:template match="/daten">
<svg xmlns="http://www.w3.org/2000/svg" version="1.0">

  <xsl:for-each select="datensatz">
    <xsl:variable name="aktuelle_zeit" select="zeit"/>
    <xsl:variable name="aktueller_wert" select="wert"/>
    <xsl:variable name="satznummer" select="position()"/>
    <xsl:variable name="rect_xpos" select="20+90*($satznummer - 1)"/>
    <xsl:variable name="rect_ypos" select="200-$aktueller_wert"/>
    <rect x="{$rect_xpos}" y="{$rect_ypos}
          width="40" height="{$aktueller_wert}" fill="green"/>
    <text x="{$rect_xpos}" y="220" font-size="14">
          <xsl:value-of select="$aktuelle_zeit"/>
    </text>
    <text x="{$rect_xpos}" y="{$rect_ypos}" font-size="14">
          <xsl:value-of select="$aktueller_wert"/>
    </text>
  </xsl:for-each>

  <line x1="0" y1="200" x2="200" y2="200" stroke="black" stroke-width="3
</svg>
```

Result Scalable Vector Graphics

```
<?xml version="1.0" encoding="UTF-8"?>

<svg xmlns="http://www.w3.org/2000/svg"
      version="1.0">

<rect x="20" y="60" width="40" height="140"
      fill="green"/>

<text x="20" y="220" font-size="14"> März
</text>

<text x="20" y="60" font-size="14"> 140
</text>

<rect x="110" y="30" width="40" height="170"
      fill="green"/>

<text x="110" y="220" font-size="14"> April
</text>

<text x="110" y="30" font-size="14"> 170
</text>

<line x1="0" y1="200" x2="200" y2="200"
      stroke="black" stroke-width="3"/>

</svg>
```

XML Overview – In Practise

- XML is able to separate
  - Content
  - Structure and
  - Format

  of a document
- XML can describe
  - Data and
  - Documents

  comparably
- Most important:
  XML can be created and processed automatically with outstanding ease

XML Overview

- Potential disadvantages of XML:
  - Babylonian confusion –
    each and everyone is creating his/her own vocabulary
  - The big number of XML based standards
    impedes overview and usage

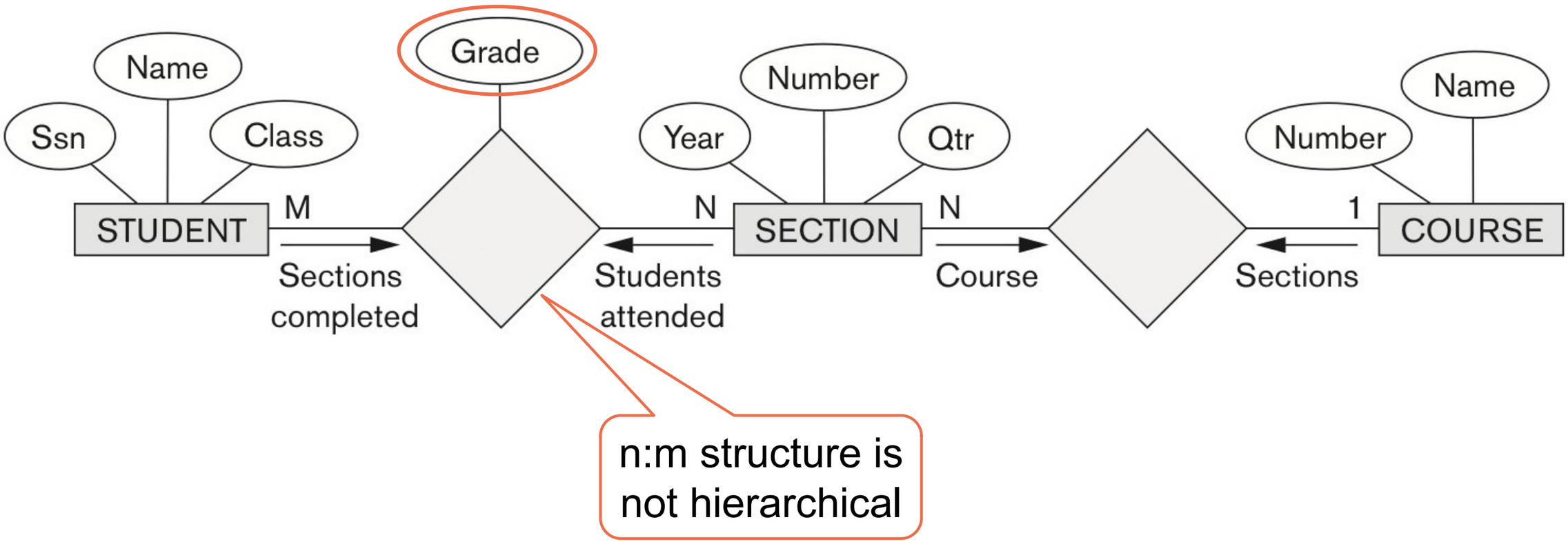
XML and Databases
Storing and Extracting XML from Databases

- **Using a DBMS to store the documents as text**
  Works for all XML Documents. Database needs to be able to work with documents
- **Using a DBMS to store the document contents as data elements**
  XML document needs to follow a DTD or Schema. The transfer between XML and database is done by mapping algorithms that are part of the DBMS or a specialized middleware
- **Using a Native XML DBMS**
  These systems store tree structured data. Indexing, querying and compression are specialized to that structure. Examples: BaseX (BSD) and webMethods Tamino (Software AG)

Details see [ElNa16] 472

Converting relational Database Data to XML

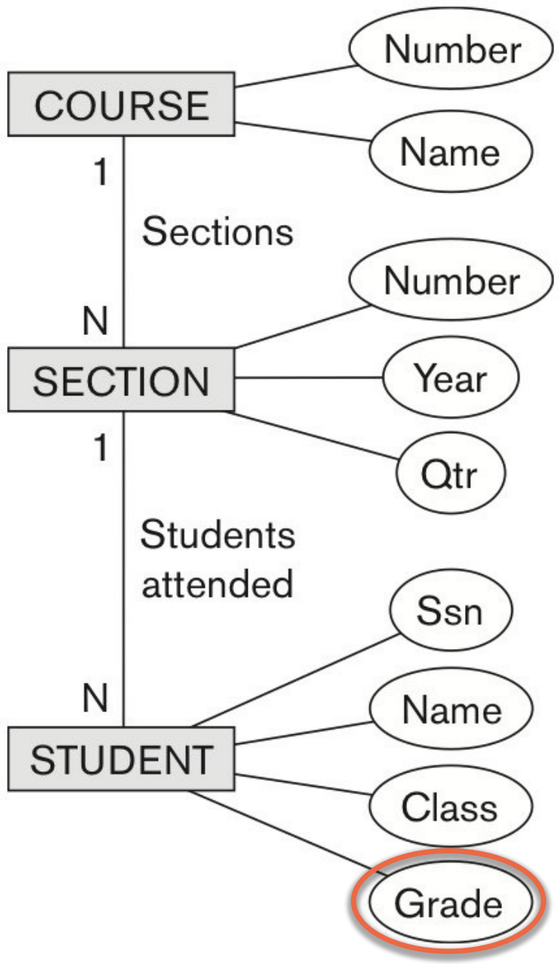
- The relational structure that should be converted



[ElNa16] 479

Converting relational Database Data to XML

- Solution 1: Course as root

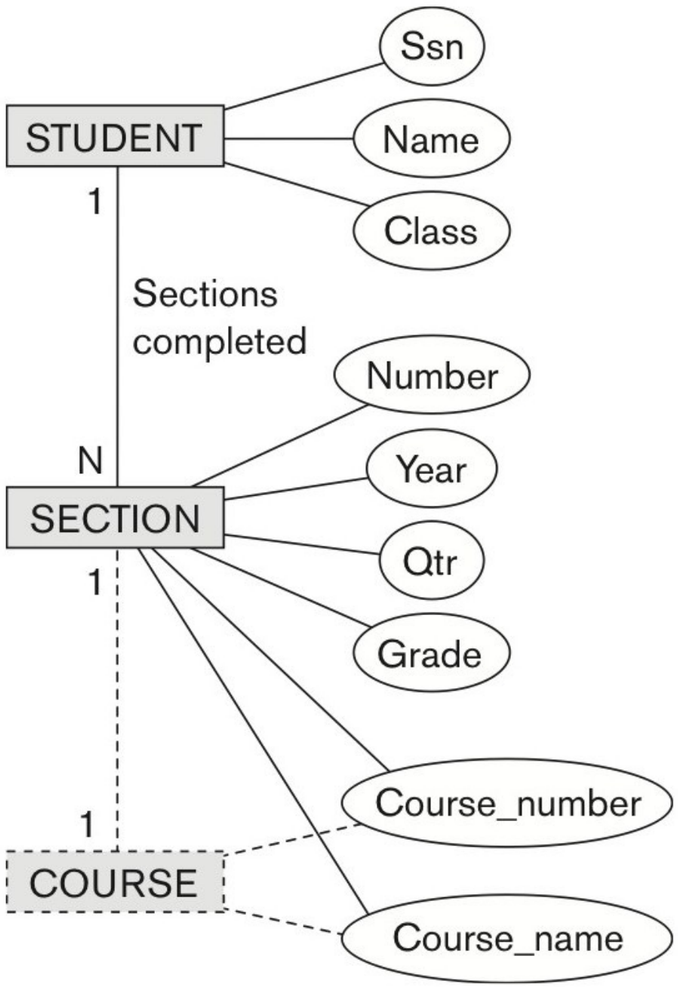



```
<xsd:element name="root">
  <xsd:sequence>
  <xsd:element name="course" minOccurs="0" maxOccurs="unbounded">
    <xsd:sequence>
      <xsd:element name="cname" type="xsd:string" />
      <xsd:element name="cnumber" type="xsd:unsignedInt" />
      <xsd:element name="section" minOccurs="0" maxOccurs="unbounded">
        <xsd:sequence>
          <xsd:element name="secnumber" type="xsd:unsignedInt" />
          <xsd:element name="year" type="xsd:string" />
          <xsd:element name="quarter" type="xsd:string" />
          <xsd:element name="student" minOccurs="0" maxOccurs="unbounded">
            <xsd:sequence>
              <xsd:element name="ssn" type="xsd:string" />
              <xsd:element name="sname" type="xsd:string" />
              <xsd:element name="class" type="xsd:string" />
              <xsd:element name="grade" type="xsd:string" />
            </xsd:sequence>
          </xsd:element>
        </xsd:sequence>
      </xsd:element>
    </xsd:sequence>
  </xsd:element>
  </xsd:sequence>
</xsd:element>
```

[ElNa16] 479

Converting relational Database Data to XML

- Solution 2: Student as root

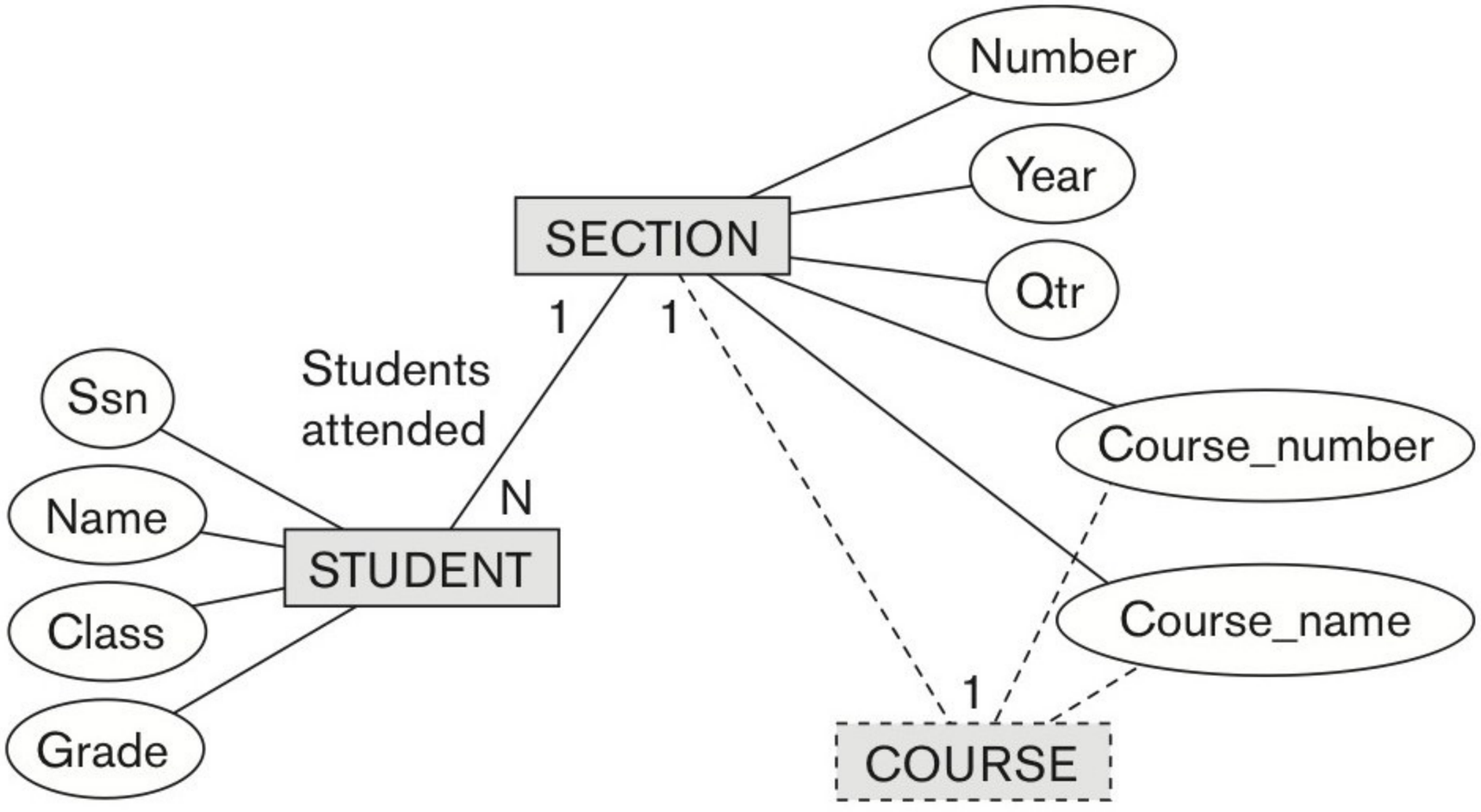



```
<xsd:element name="root">
<xsd:sequence>
<xsd:element name="student" minOccurs="0" maxOccurs="unbounded">
    <xsd:sequence>
        <xsd:element name="ssn" type="xsd:string" />
        <xsd:element name="sname" type="xsd:string" />
        <xsd:element name="class" type="xsd:string" />
        <xsd:element name="section" minOccurs="0" maxOccurs="unbounded">
            <xsd:sequence>
                <xsd:element name="secnumber" type="xsd:unsignedInt" />
                <xsd:element name="year" type="xsd:string" />
                <xsd:element name="quarter" type="xsd:string" />
                <xsd:element name="cnumber" type="xsd:unsignedInt" />
                <xsd:element name="cname" type="xsd:string" />
                <xsd:element name="grade" type="xsd:string" />
            </xsd:sequence>
        </xsd:element>
    </xsd:sequence>
</xsd:element>
</xsd:sequence>
</xsd:element>
```

[ElNa16] 481

Converting relational Database Data to XML

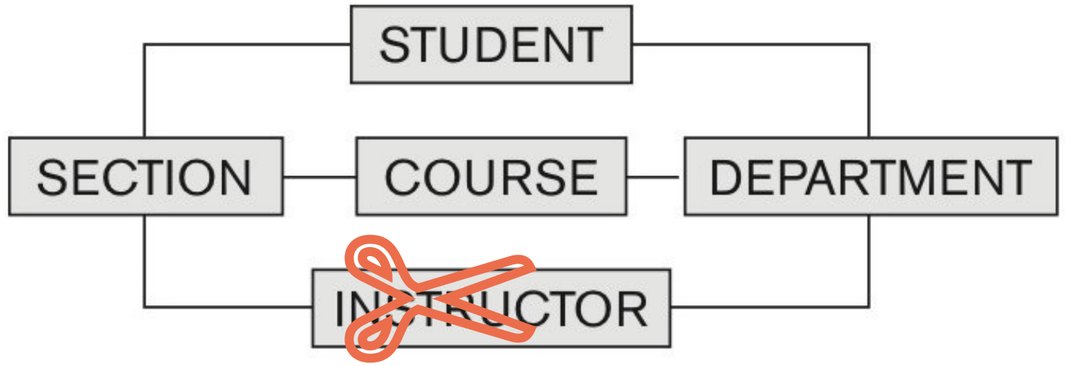
- Solution 3: Section as root



[ElNa16] 482

Converting relational Database Data to XML

- Structure with circle

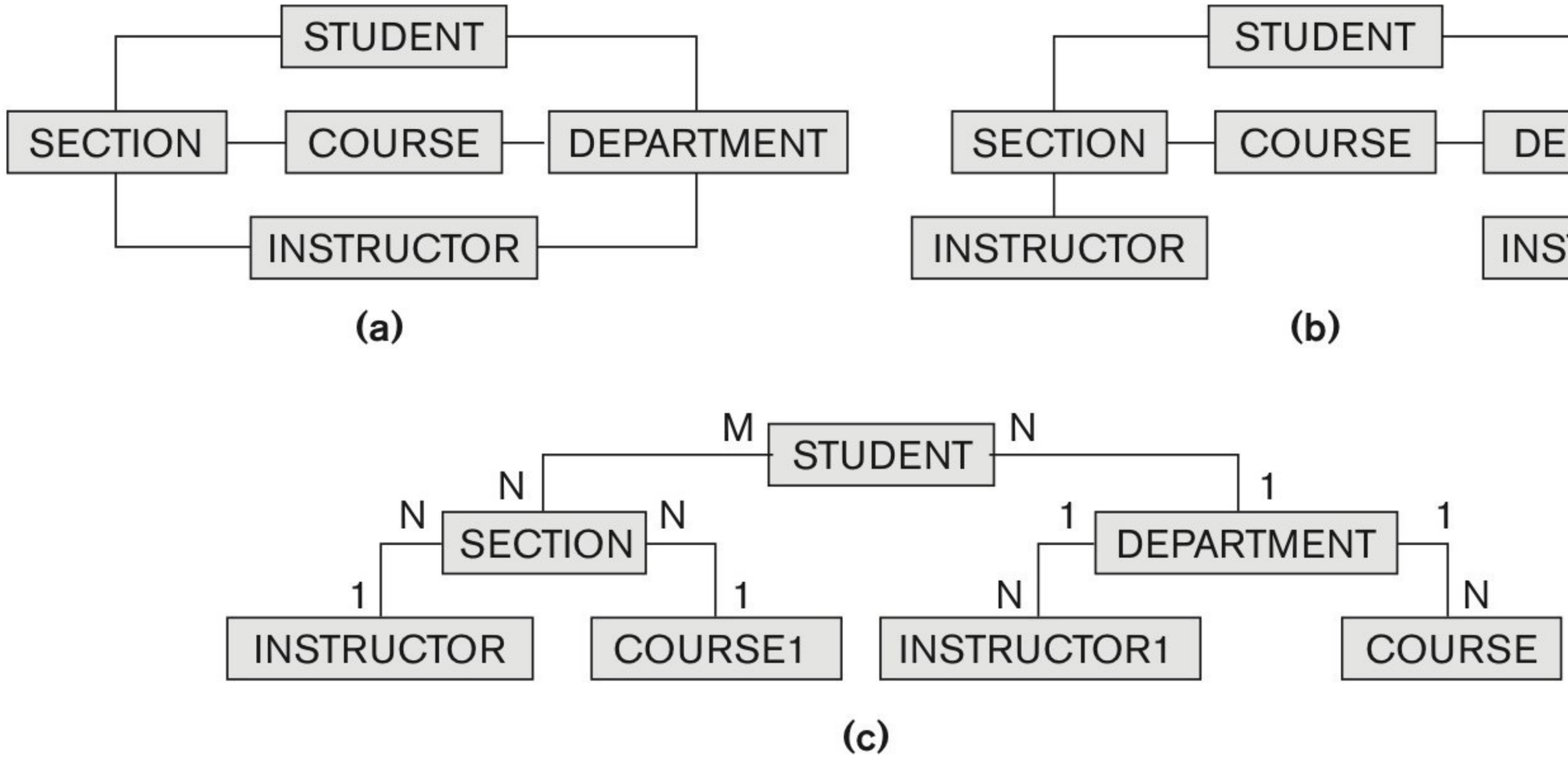


(a)

[ElNa16] 482

Converting relational Database Data to XML

- Structure with circles



[ElNa16] 482

Creating XML in SQL 1

- Create a table in your database using the following code (Car_create_data.sql provided in ILIAS)

```
DROP TABLE IF EXISTS Car;

CREATE TABLE Car
(
  CarId int identity(1,1) primary key,
  Name varchar(100),
  Make varchar(100),
  Model int ,
  Price int ,
  Type varchar(20)
);
```

```
INSERT INTO Car( Name, Make,  Model ,
Price, Type)
VALUES ('Corrolla','Toyota',2015,
20000,'Sedan'),
('Civic','Honda',2018, 25000,'Sedan'),
('Passo','Toyota',2012, 18000,'Hatchback'),
('Land Cruiser','Toyota',2017,
40000,'SUV'),
('Corrolla','Toyota',2011, 17000,'Sedan'),
('Vitz','Toyota',2014, 15000,'Hatchback'),
('Accord','Honda',2018, 28000,'Sedan'),
('7500','BMW',2015, 50000,'Sedan'),
('Parado','Toyota',2011, 25000,'SUV'),
('C200','Mercedez',2010, 26000,'Sedan'),
('Corrolla','Toyota',2014, 19000,'Sedan'),
('Civic','Honda',2015, 20000,'Sedan');
```

The example is derived from https://www.sqlshack.com/working-with-xml-data-in-sql-server/

Creating XML in SQL 2

- First attempt

```
SELECT * FROM Car
FOR XML AUTO;
```

Click on Link in results to see full result (after each attempt

- Second attempt with paths

```
SELECT * FROM Car
FOR XML PATH;
```

- Third attempt with root

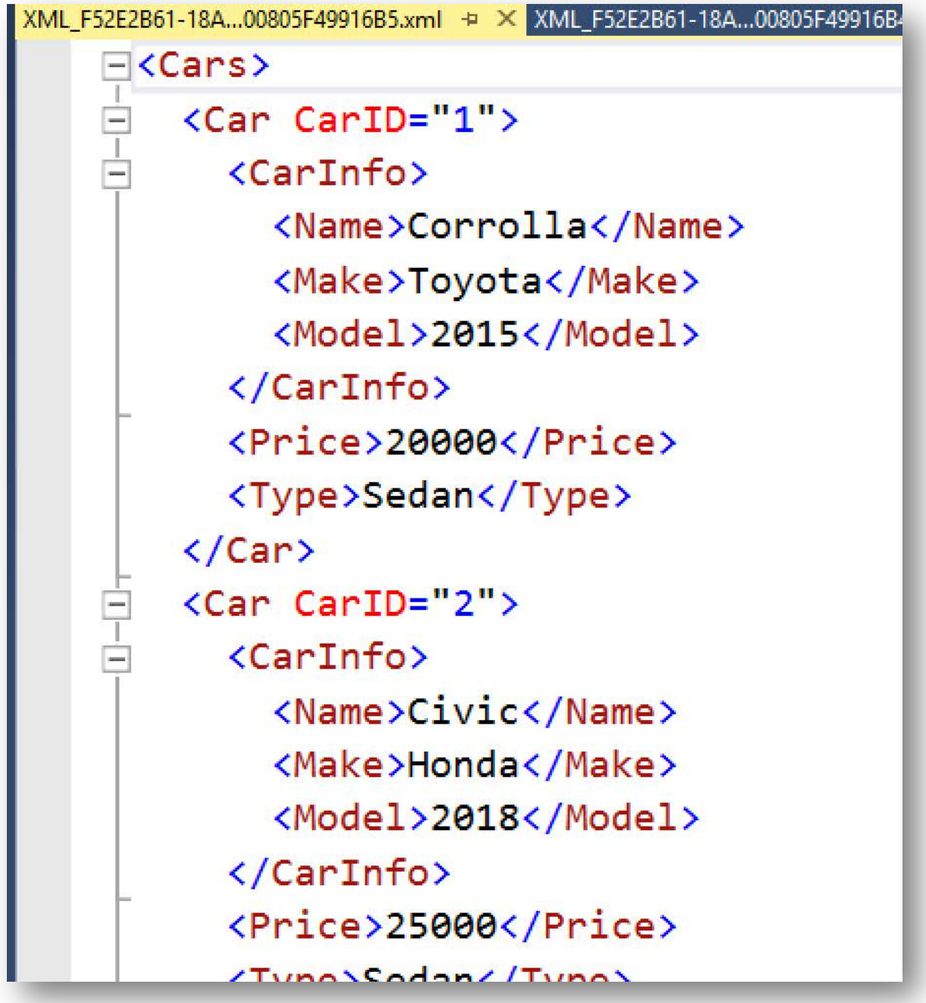
```
SELECT * FROM Car
FOR XML PATH ('car'), ROOT ('cars');
```

Practice

Creating XML in SQL 3

- Nesting and renaming the data

```
SELECT  CarId as [@CarID],
        Name  AS [CarInfo/Name],
        Make [CarInfo/Make],
        Model [CarInfo/Model],
        Price,
        Type
FROM Car
FOR XML PATH ('Car'), ROOT('Cars')
```



Practice

Creating XML in SQL 4

- Giving it more structure

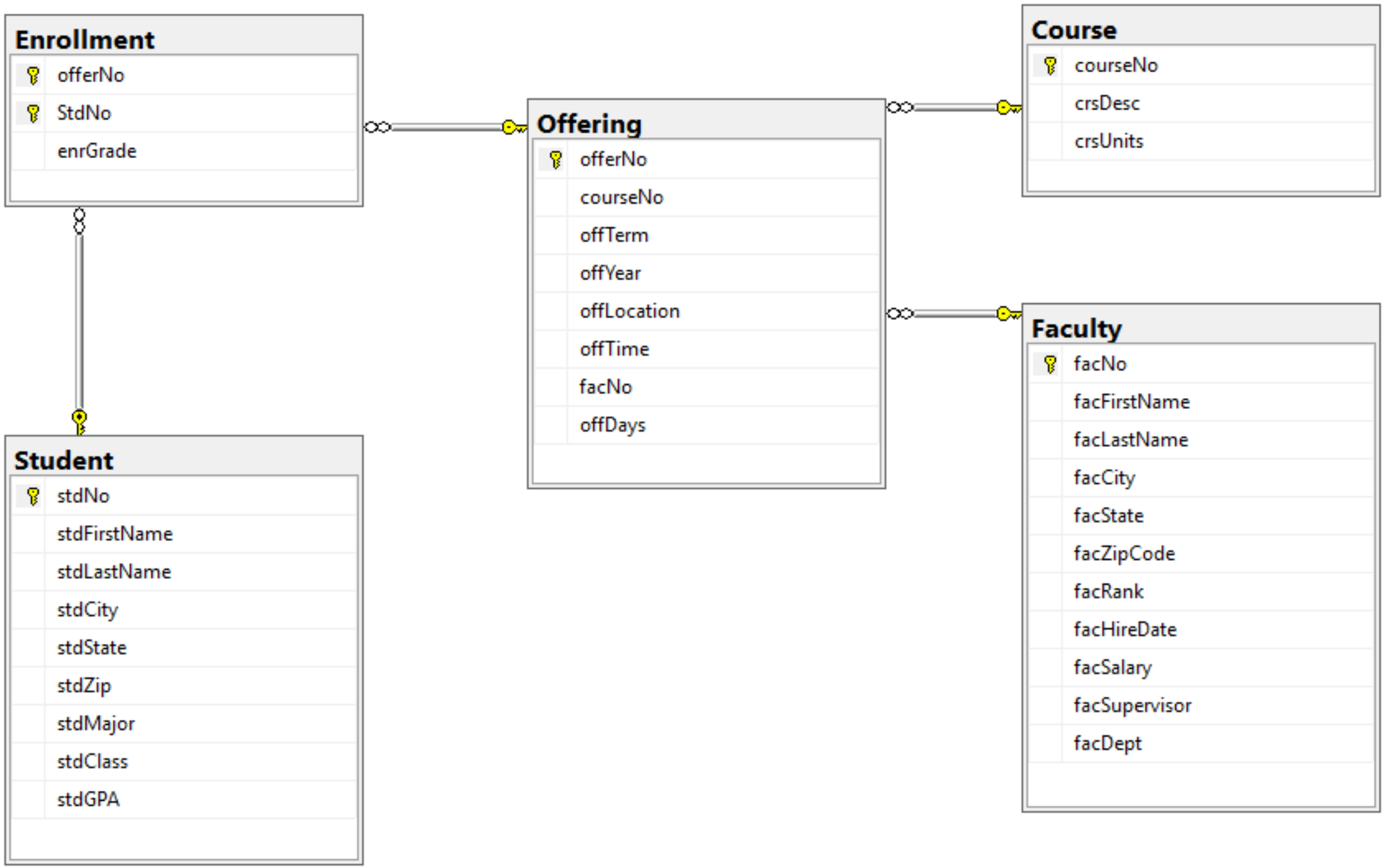
```
SELECT  CarId as [@CarID],
        Name  AS [CarInfo/@Name],
        Make [CarInfo/@Make],
        Model [CarInfo/Model],
        Price,
        Type
FROM Car
FOR XML PATH ('Car'), ROOT('Cars')
```

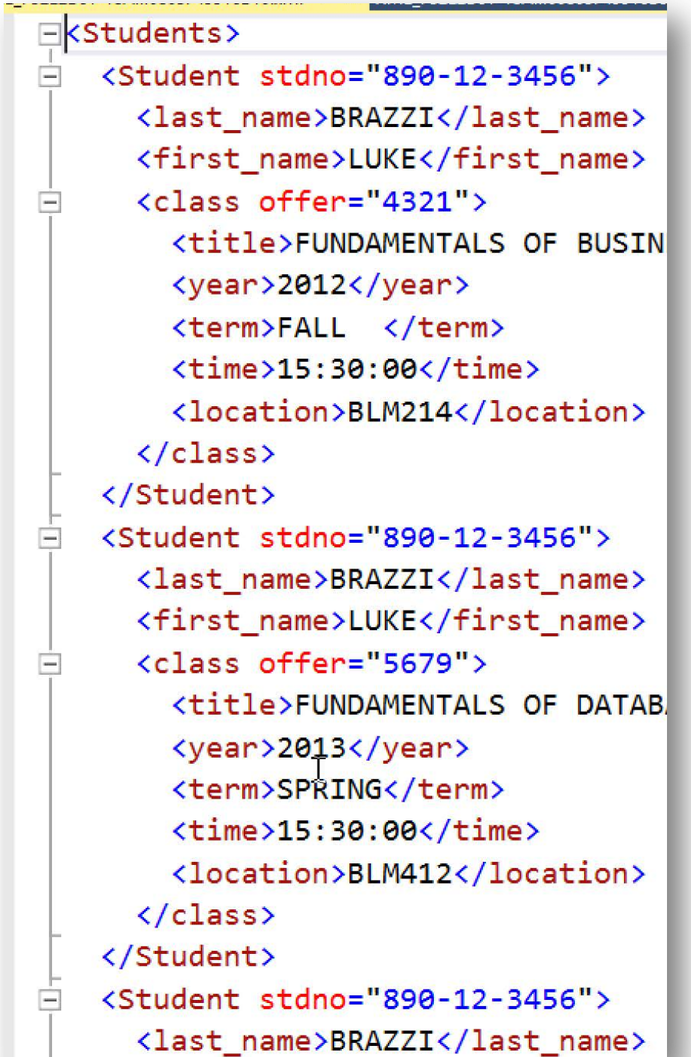
```
<Cars>
  <Car CarID="1">
    <CarInfo Name="Corrolla" Make="Toyota">
      <Model>2015</Model>
    </CarInfo>
    <Price>20000</Price>
    <Type>Sedan</Type>
  </Car>
  <Car CarID="2">
    <CarInfo Name="Civic" Make="Honda">
      <Model>2018</Model>
    </CarInfo>
    <Price>25000</Price>
    <Type>Sedan</Type>
  </Car>
  <Car CarID="3">
    <CarInfo Name="Passo" Make="Toyota">
      <Model>2012</Model>
    </CarInfo>
    <Price>18000</Price>
    <Type>Hatchback</Type>
  </Car>
  <Car CarID="4">
```

Practice

How can we create an XML-listing of all offerings per student (Student name, crsDesc, time/loc)?

27 November 2023





```
<Students>
  <Student stdno="890-12-3456">
    <last_name>BRAZZI</last_name>
    <first_name>LUKE</first_name>
    <class offer="4321">
      <title>FUNDAMENTALS OF BUSIN
      <year>2012</year>
      <term>FALL  </term>
      <time>15:30:00</time>
      <location>BLM214</location>
    </class>
  </Student>
  <Student stdno="890-12-3456">
    <last_name>BRAZZI</last_name>
    <first_name>LUKE</first_name>
    <class offer="5679">
      <title>FUNDAMENTALS OF DATAB
      <year>2013</year>
      <term>SPRING</term>
      <time>15:30:00</time>
      <location>BLM412</location>
    </class>
  </Student>
  <Student stdno="890-12-3456">
    <last_name>BRAZZI</last_name>
```

Practice

How can we create an XML-listing of all offerings per student (Student name, crsDesc, time/loc)?

27 November 2023

```
SELECT s.stdNo AS [@stdno],
  stdLastName AS [last_name],
  stdFirstName AS [first_name],
  o.offerNo AS [class/@offer],
  crsDesc AS [class/title],
  offYear AS [class/year],
  offTerm AS [class/term],
  offTime AS [class/time],
  offLocation AS [class/location]
  FROM Student s, Enrollment e, Offering o,
  Course c
  WHERE s.StdNo=e.StdNo and e.offerNo=o.offerNo
  and o.courseNo=c.courseNo
  ORDER BY stdLastName, stdFirstName, offYear,
  offTerm
  FOR XML PATH ('Student'), ROOT ('Students')
```

```
<Students>
  <Student stdno="890-12-3456">
    <last_name>BRAZZI</last_name>
    <first_name>LUKE</first_name>
    <class offer="4321">
      <title>FUNDAMENTALS OF BUSIN
      <year>2012</year>
      <term>FALL  </term>
      <time>15:30:00</time>
      <location>BLM214</location>
    </class>
  </Student>
  <Student stdno="890-12-3456">
    <last_name>BRAZZI</last_name>
    <first_name>LUKE</first_name>
    <class offer="5679">
      <title>FUNDAMENTALS OF DATAB
      <year>2013</year>
      <term>SPRING</term>
      <time>15:30:00</time>
      <location>BLM412</location>
    </class>
  </Student>
  <Student stdno="890-12-3456">
    <last_name>BRAZZI</last_name>
```

Practice

How can we create an XML-listing of all offerings per student? → Nested version


```
SELECT s.stdNo AS '@stdno',
  stdLastName AS 'last_name',
  stdFirstName AS 'first_name',
    (SELECT offYear AS 'year',
    offTerm AS 'term',
       (SELECT crsDesc AS 'title',
       offTime AS 'time',
       offLocation AS 'location'
       FROM Offering o, Course c
       WHERE  o.courseNo=c.courseNo and
       e.offerNo=o.offerNo
       FOR XML PATH ('')) as 'course'
   FROM  Enrollment e, Offering o
   WHERE s.StdNo=e.StdNo and
   e.offerNo=o.offerNo
   FOR XML PATH ('')) AS 'semester'
FROM Student s
ORDER BY stdLastName, stdFirstName
FOR XML PATH ('student'), ROOT ('students')
```

```
▼<students>
 ▼<student stdno="890-12-3456">
    <last_name>BRAZZI</last_name>
    <first_name>LUKE</first_name>
   ▼<semester>
      <year>2012</year>
      <term>FALL </term>
     ▼<course>
        <title>FUNDAMENTALS OF BUSINESS PROGRAMMING</title>
        <time>15:30:00</time>
        <location>BLM214</location>
      </course>
      <year>2013</year>
      <term>SPRING</term>
     ▼<course>
        <title>FUNDAMENTALS OF DATABASE MANAGEMENT</title>
        <time>15:30:00</time>
        <location>BLM412</location>
      </course>
      <year>2013</year>
      <term>SPRING</term>
     ▼<course>
        <title>CORPORATE FINANCE</title>
        <time>13:30:00</time>
        <location>BLM305</location>
      </course>
    </semester>
  </student>
 ▼<student stdno="876-54-3210">
    <last_name>COLAN</last_name>
    <first_name>CRISTOPHER</first_name>
```

Recommendation
Working with XMLSpy

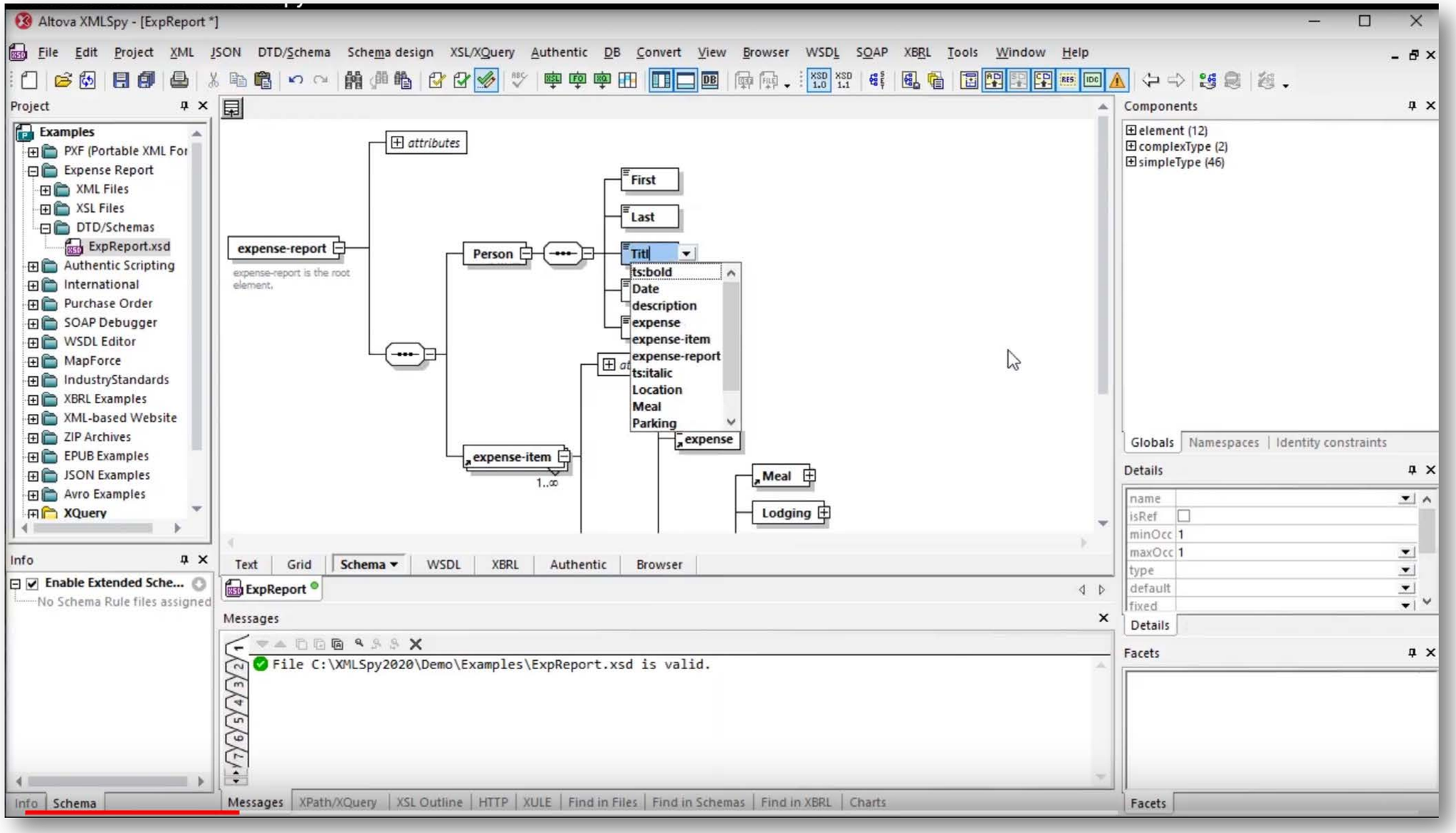
Altova XMLSpy allows developers to create XML-based and Web services applications.

It includes tools for:

- XML instance document creation and editing
- Visual XML Schema development
- DTD editing
- XSLT 1.0/2.0 development and debugging
- XQuery development and debugging
- XPath 1.0/2.0 development and analysis
- Office Open XML development
- XBRL taxonomy & instance document creation, editing, and validation
- JSON and JSON Schema editing & conversion
- …

Source: Altova

Discovering XMLSpy



https://www.youtube.com/watch?v=QJZGawmLu58

Practice

Seminar – Todo (optional)

- Work through the XMLSpy Tutorial (link see next page) especially
  - XML Schema: Basics
  - XML Documents
  - XSLT Transformations

- Convert the file timetable_edb5_split_160930.csv to XML
- Explore the bmecat_2005_1.xsd Schema

- Try to create a XSD out of a Segment of our exercise example (the SQL training database)

For your self-studies

- About XML
  https://www.w3.org/standards/xml/core
- About XML Schema:
  http://www.w3.org/TR/xmlschema-0/
- Altova tutorial:
  www.**altova**.com/en/documents/**XMLSpyTutorial**.pdf
- Short version:
  www.**altova**.com/en/documents/**XMLSpyTutorialS**td.pdf
- Detailed information
  http://www.altova.com/whitepapers/datamodeling.pdf

Reference

[Banz20] Banzal, S.: XML basics. Duxbury: Mercury Learning and Information, 2020 — ISBN 978-1-68392-546-0

[Bech22] Becher, Margit: XML: DTD, XML-Schema, XPath, XQuery, XSL-FO, SAX, DOM. Wiesbaden: Springer Fachmedien Wiesbaden, 2022. URL https://link.springer.com/10.1007/978-3-658-35435-0 — ISBN 978-3-658-35434-3

[BKGB16] Bauer, Christian; King, Gavin; Gregory, Gary; Bauer, Christian: Java persistence with Hibernate. Second edition. Shelter Island, NY: Manning, 2016 — ISBN 978-1-61729-045-9

[Data00] Data Exchange Standards | IPC. URL https://www.ipc.org/materials-declaration-data-exchange-standards-homepage.

[Elec00] Electricity Load Forecasting Using Data Mining Technique | InTechOpen. URL http://www.intechopen.com/books/ advances-in-data-mining-knowledge-discovery-and-applications/electricity-load-forecasting-using-data-mining-technique.

[ElNa16] Elmasri, Ramez; Navathe, Shamkant B.: Fundamentals of Database Systems, Global Edition. 7th Revised edition: Pearson Education Limited, 2016 —ISBN 978-1-292-09761-9

[Etim00] ETIM Deutschland e.V. URL http://www.etim.de/home/.

[Grup18] Grupe, Wilfried: XML: Grundlagen, Technologien, Validierung, Auswertung. 1. Auflage. Frechen: mitp, 2018 — ISBN 978-3-95845-754-6

[HaMU05] Harold, Elliotte Rusty; Means, W. Scott; Udemadu, Katharina: XML in a nutshell. 3. Aufl., Ausg. Köln (u.a.): O'Reilly, 2005 — ISBN 978-3-89721-339-5

[LaLa20] Laudon, Kenneth C; Laudon, Jane Price: Management information systems: managing the digital firm. 16: Pearson Education Canada, 2020 — ISBN 978-1-292-29656-2

[Math00] Mathematical Markup Language (MathML 1.01) DTD. URL https://www.w3.org/TR/REC-MathML/appendixA.html.

[Shas21] Shashi Bhushan: Object/Relational Impedance Mismatch. URL https://medium.com/booleanbhushan/object-relational-impedance-mismatch-28fc2440dee4. — Medium.com

[Xmle00] XML Essentials - W3C. URL https://www.w3.org/standards/xml/core.